挑战5：裸机下驱动蓝牙（Bluetooth）模块

模块目标

实现蓝牙的输入输出。

（具体的放到后面再说吧 咳咳）

模块实现

> 因为我们的 HM-15 很不幸坏掉了，所以这里是拿两块 HM-10 来连接的（咳咳）

第一块 HM-10 作为无人机上的蓝牙从机，接在开发板上，被动等待连接；
第二块 HM-10 作为设备上的蓝牙主机，通过 USB-TTL 串口模块接在电脑上，主动发出连接。

蓝牙原理

HM-10 是一个已经封装好了的蓝牙通信模块，我们只需要利用其暴露出的串口进行串口通信，即可接受或向其发送信息。

HM-10 共有以下引脚：

HM-10 芯片引脚	**作用**	**逻辑**
STATE	状态指示引脚，用来指示蓝牙当前的连接状态	通常模块未连接时输出低电平（0V），一旦手机或其他设备连上模块，它会输出高电平（3.3V）。*可悬空不接*
VCC	电源正极，给模块供电	不可以接入5V，请从STM32开发板上的3.3V引脚引线过来
GND	电源负极/地，接地	必须和STM32开发板共地（连在一起），这是UART通信的基础
TXD	发送引脚，模块向外发送数据（Transmit Data）	模块的 TXD 要接到 STM32 的 USART_RX 引脚（交叉接线）
RXD	接收引脚，模块接收外部发来的数据（Receive Data）	模块的 RXD 要接到 STM32 的 USART_TX 引脚（交叉接线）
BRK	断开/中断引脚，Break/Disconnect（断开连接）	主要用于主动切断当前的蓝牙连接，或者强制模块退出透传模式进入AT指令模式。*可悬空不接*

接线完成后，开发板与蓝牙模块的串口通信就已经搭建好了，接下来就需要进行蓝牙连接，让蓝牙模块能连上我们的设备（手机/电脑）

在连接蓝牙之前，我们需要设置并记录一下手上两颗 HM-10 模块的参数。
使用 USB-TTL 串口模块直接连接 HM-10 模块，在电脑的串口助手中使用 `AT` 指令进行设置/读取，记录如下：

从机参数

记录项	记录值	备注	指令
HM-10 MAC 地址	6CB2FD05064B	主机可以依据这个地址连接它	`AT+ADDR?`
蓝牙名称	HMSoft	连接时显示的设备名称（默认的，没改）	`AT+NAME?`
当前波特率	9600	STM32 的 USART1 初始化必须用这个值	`AT+BAUD0` `AT+BAUD?`
固件版本	`HMSoft V610`	比如 V610 或 V707	`AT+VERS?`
模块角色	从机（ROLE=0）	无人机端固定从机	`AT+ROLE0` `AT+ROLE?`
工作模式	透传（MODE=0）	收到什么发什么	`AT+MODE0` `AT+MODE?`
服务 UUID	0xFFE0	手机 APP 连接时看得到（保持默认即可）	默认值，不修改不查询
特征 UUID	0xFFE1	读写数据全靠它（保持默认即可）	默认值，不修改不查询

设置并记录完成后，即可进行开发板接线了。

主机参数

记录项	记录值	备注	指令
蓝牙名称	`Ground`（建议修改）	建议改名为 Ground 以区分无人机端	`AT+NAME` `Ground` `AT+NAME?`
当前波特率	9600	⚠️核心：必须与从机完全一致！	`AT+BAUD0` `AT+BAUD?`
固件版本	`HMSoft V610`	记录备用，最好与从机一致	`AT+VERS?`
模块角色	主机（ROLE=1）	⚠️核心：主动去连从机	`AT+ROLE1` `AT+ROLE?`
工作模式	透传（MODE=0）	收到什么发什么，不需要AT指令干预	`AT+MODE0` `AT+MODE?`
服务 UUID	0xFFE0	默认值，不做修改	默认值
特征 UUID	0xFFE1	默认值，不做修改	默认值

> ⚠ **Warning**

> 在你没做好准备之前，不要随便输入 `AT+ROLE1` 和 `AT+MODE0` ，除非你能保证周围环境里一个正在运行的蓝牙设备都没有！
> 因为主机模式会自动搜索附近的可连接蓝牙，然后自动连接，如果你设置了 `AT+MODE0` 透传模式，然后又随机连到了什么不知名的设备去，它就会开机自动连接、保持连接、并且不听你的指令，就炸了……

建议是先 `AT+MODE2` ，别着急开透传，然后在其他模式下先连上，再进一步设置 `AT+MODE0` 。

看看参考手册原文：

> 0: 透传模式　1: PIO 采集+远控+透传 2：透传+远控模式
>
> **透传模式**：即普通的串口透明传输,蓝牙通到数据后转发至串口，同时也 转发串口收到的数据到远端蓝牙。
>
> **PIO 采集+远控+透传模式**：在此模式下，远端蓝牙可以使用AT指令设置模 块信息，还可以远程控制2路PIO输出状态(PIO2,PIO3)，同时可以采集8路 PIO 状态(PIO4~PIO11)，同时还可以将不是“AT”打头的数据包转发至串口 进行双向通讯。
>
> **透传+远控模式**：在此模式下，远端蓝牙可以使用AT指令设置模块信息，还 可以控制10 路PIO(PIO2~PIO11)输出状态，同时还可以将非“AT”开头的 数据包转发至串口进行双向通讯。

然后建议再 `AT+IMME1` 设为手动模式，然后手动连接，不然自动连接很容易连到其他设备去。

看看参考手册原文：

> 这是一个重要指令，用于切换模块的自动，手动工作模式。
>
> 0: 上电立即自动工作
> 1: 上电后暂不工作 等待AT+START; AT+CON; AT+CONN; AT+CO; AT+DISI?; AT+DISC?等指令

HM-10 自动主模式没有 “ 只认某一个 MAC 地址自动连接 ” 的内置功能； `AT+ROLE2` 会连信号最强的从机， `AT+ROLE1 + AT+IMME0` 也会按搜索列表尝试连接，容易误连其他 HM-10。

干完这些以后再 `AT+ROLE1` ，然后根据参考手册上的：

先 `AT+DISC?` ，扫描从机；
再从返回的列表里，找到我们的 HMSoft（确保在列表里，在才能连得上）；
最后再 `AT+CO06CB2FD05064B` 就连上了。

> 这个指令其实分为三个部分：
>
AT+CO	0	6CB2FD05064B
连接指定 MAC 地址的从设备指令	代表接下来给它的是静态地址	从机的 MAC 地址

这是第一次连接，以后就可以直接 `AT+CONNL` 连接最后一次连接成功的从设备。【手册原文：该指令为主模式、手动工作模式指令。使用前须满足 AT+ROLE1， AT+IMME1。若模块没有记住最后一次成功连接过设备地址，则不响应。】

代码实现

然后我们回到 Keil 去写 STM32的代码，实现无人机上的收发。

初始化

```
// 启动 GPIOA 和 USART2 的时钟
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART2, ENABLE);
    RCC_APB2PeriphClockCmd(RCC_APB2Periph_GPIOA, ENABLE);

    GPIO_InitTypeDef GPIO_InitStructure;
    GPIO_InitStructure.GPIO_Speed = GPIO_Speed_50MHz;
// PA2 (TX) 配置为复用推挽输出
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_2;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_AF_PP;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
// PA3 (RX) 配置为浮空输入
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_3;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IN_FLOATING;
    GPIO_Init(GPIOA, &GPIO_InitStructure);
// PA6 (STATE) 配置为上拉输入
    GPIO_InitStructure.GPIO_Pin = GPIO_Pin_6;
    GPIO_InitStructure.GPIO_Mode = GPIO_Mode_IPU;
    GPIO_Init(GPIOA, &GPIO_InitStructure);

// USART 初始化 (9600、输入输出、无流控、无校验、8位长、停止位1位)
    USART_InitTypeDef USART_InitStructure;
    USART_InitStructure.USART_BaudRate = 9600;
    USART_InitStructure.USART_Mode = USART_Mode_Tx | USART_Mode_Rx;
    USART_InitStructure.USART_HardwareFlowControl =
USART_HardwareFlowControl_None;
    USART_InitStructure.USART_Parity = USART_Parity_No;
    USART_InitStructure.USART_WordLength = USART_WordLength_8b;
    USART_InitStructure.USART_StopBits = USART_StopBits_1;
    USART_Init(USART2, &USART_InitStructure); // 初始化串口
    USART_Cmd(USART2, ENABLE); // 启动串口（别忘了这个Cmd）
```

没看懂的点击这里可以跳转到串口初始化的知识点汇总。

蓝牙输出

蓝牙的输出指的是我们把数据从开发板上通过串口通信传给蓝牙模块（我们的代码控制的是这个部分），然后蓝牙模块拿着这个数据通过蓝牙再传给它连接着的设备（这个部分由 HM-10 模块自己完成，我们管不着）

```
    while(USART_GetFlagStatus(USART2, USART_FLAG_TXE) == RESET); // 等待TXE置1
    USART_SendData(USART2, data); // 向蓝牙模块传输数据data
```

详细解释看这里。

蓝牙输入

蓝牙的输入指的是我们把数据从设备（电脑/手机）上发送出去，蓝牙模块自动接收到以后，会把这个数据从 HM-10 模块里以串口通信的方法向我们的开发板传输。我们需要处理的就是从串口里传回来的这些数据。

常用两种方法：一是使用 NVIC 中断，一是使用 DMA 转移数据。

我们先看第一种：中断处理法。

先要在初始化中添加一下对 NVIC（嵌套向量中断控制器）的配置：

```
// 第一步：在外设层面使能中断
    USART_ITConfig(USART2, USART_IT_RXNE, ENABLE);
// 含义：当USART2的RXNE（接收数据寄存器非空）事件发生时，向NVIC发出中断请求

// 第二步：在NVIC层面使能中断通道并设置优先级
    NVIC_InitTypeDef NVIC_InitStructure;
    NVIC_InitStructure.NVIC_IRQChannel = USART2_IRQn;            // 选择USART2的中断
通道
    NVIC_InitStructure.NVIC_IRQChannelPreemptionPriority = 1;    // 抢占优先级
    NVIC_InitStructure.NVIC_IRQChannelSubPriority = 1;           // 子优先级
    NVIC_InitStructure.NVIC_IRQChannelCmd = ENABLE;              // 使能这个通道
    NVIC_Init(&NVIC_InitStructure);
```

NVIC 的详解可以看这里。

随后，我们需要定义**中断函数**。

查阅启动文件（startup_stm32f10x_md.s）中的中断向量表可知：

```
                DCD     USART2_IRQHandler          ; USART2
                EXPORT  USART2_IRQHandler          [WEAK]
```

中断函数名称必须和启动文件中的向量表一致，故而：

```
void USART2_IRQHandler(void) {
    // 判断是哪个事件触发了中断
    if(USART_GetITStatus(USART2, USART_IT_RXNE) != RESET) {
        uint16_t data = USART_ReceiveData(USART2); // 读取数据（读DR会自动清除RXNE标
志）
        // 在这里写你的业务逻辑
    }
}
```

便可以处理输入：在获得了输入时会触发中断，CPU 就会停下手里的活过来处理我们的输入，处理完了再回去继续之前的活。

注意：虽然我们读取到了16位的数据，但是有效位只有前8或9位，其他的高位均为0！

实现包通信

为什么我们需要包通信呢?

因为我们的接收、发送，中间都会遇到很多的干扰，导致数据丢失的现象出现。

比如两个浮点数1.2、3.4，它们的 IEEE 754格式是 `0x3F99999A` 、 `0x4059999A` 。
我们在发送这两个数的时候，正常应该是这样的八个字节：

3F	**99**	**99**	**9A**	**40**	**59**	**99**	**9A**

如果我们恰好从3F 开始接收，就可以四个四个的定长解析，正常得到 `3F99999A` 、 `4059999A` 。

可是如果中间出错了，漏了一个字节呢?

3F	99	9A	40	59	99	9A

又或者

3F	99	99	9A	40	99	9A

在事先没法知道浮点数的情况下，我们怎么知道漏掉的是哪一个浮点数的字节呢？（7个字节，怎么知道应该读取前4个还是后4个？）
而且，一个出错，会影响后面的，后面的全部一起错掉，收到的数据就全是乱码了。

所以，我们需要定义包的格式（这就是协议了），用包来通信，这样我们才能尽可能的缩小丢数据的代价。

制定协议

> 协议不是固定的，这里只是我自己定的做个示例 Orz

比如，我们规定包头为 `0xAA`，中间4字节存放 float，最后一个字节存放一位的校验位，那我们的包格式如下:

1（包头）	2	3	4	5	6
0xAA	float 数据	float 数据	float 数据	float 数据	异或校验位

包总大小定长6字节，数据定长4字节。

包头

包头为 `0xAA`，意味着我们一旦收到 `0xAA` 这个字节，我们就要开始接收包了。

那万一中间的数据里也出现了 `0xAA`，难道不会冲突吗?

所以说才需要状态机。一旦之前接收到了 `0xAA`，就已经进入了接收 float 数据的状态，那么我们就不会再关注 float 内部的 `0xAA`，只做接收、保存、解析。

那要是包头丢了，float 数据中刚好出现了 `0xAA`，那咋办?

这就是校验位的作用了。

数据

数据存储时有一个大小端的问题，没搞好的话就会导致接收到的数据没法正确解析。

- **大端模式**（Big-endian）：高位字节存储在低地址，低位字节存储在高地址。
- **小端模式**（Little-endian）：低位字节存储在低地址，高位字节存储在高地址。

这里举个例子来解释大端和小端

在 STM32（小端 CPU）上，`float num = 8.1f;`（0x4101 9994）在内存中的4个字节是:

低地址	→	→	高地址
0x94	0x99	0x01	0x41

`memcpy` 是一个 C 标准库函数，用于将数据从一个内存位置复制到另一个内存位置。它本身并不关心数据的字节顺序（即大端或小端），而是直接按字节复制数据。
我们在 STM32上使用 `memcpy`，就会把 STM32的小端模式拷贝进去数组，得到：

arr[0]	arr[1]	arr[2]	arr[3]	arr[4]	arr[5]
0xAA（包头）	0x94	0x99	0x01	0x41	0x4D（校验位）

然后呢，我们如果按照大端进行接收，就会得到0x94990141，然后就炸了。
需要按照小端接收，即 arr[4] 反而要放在开头，arr[1] 放在结尾，才能得到0x41019994的正确 float 数。
所以需要在接收的时候注意一下字节的顺序。

校验位

我们这里暂时使用一些最最最简单的校验方法：异或校验。

就是把所有的数据全部异或在一起就完了。

```
/*
校验位 = 0;    //  初始化一下
校验位 ^= arr[1];
校验位 ^= arr[2];
校验位 ^= arr[3];
校验位 ^= arr[4];
                //  这样就得到校验位了
*/
```

然后接收的时候也进行一下同样的操作，最后比比看自己算出来的校验位是否和接收到的校验位一样，一样则说明大概率是没问题了，不一样就丢包。

有两个作用：

1. 防止说中间的 float 数据传错了（某些0收到干扰传成了1，或者1被干扰成了0）；
2. 防止把数据中间的 `0xAA` 当成包头：因为你找错了包头的话，一路错下去错到校验位，大概率也是无法通过的。

为什么没有包尾?

因为我们已经定长 + 校验了，有没有包尾其实并不重要。

况且，如果我们加上了包尾，接收的状态机就得加一个说：在接收完了 float 和校验位了以后，就要开始等待包尾出现，在此期间接收到的所有数据都会丢弃，直到接收到包尾。
那万一这个包的包尾丢了，然后它一直等一直丢，丢到下一个包的包尾，那我们不是白白损失了一个包吗?

那便去掉好了。

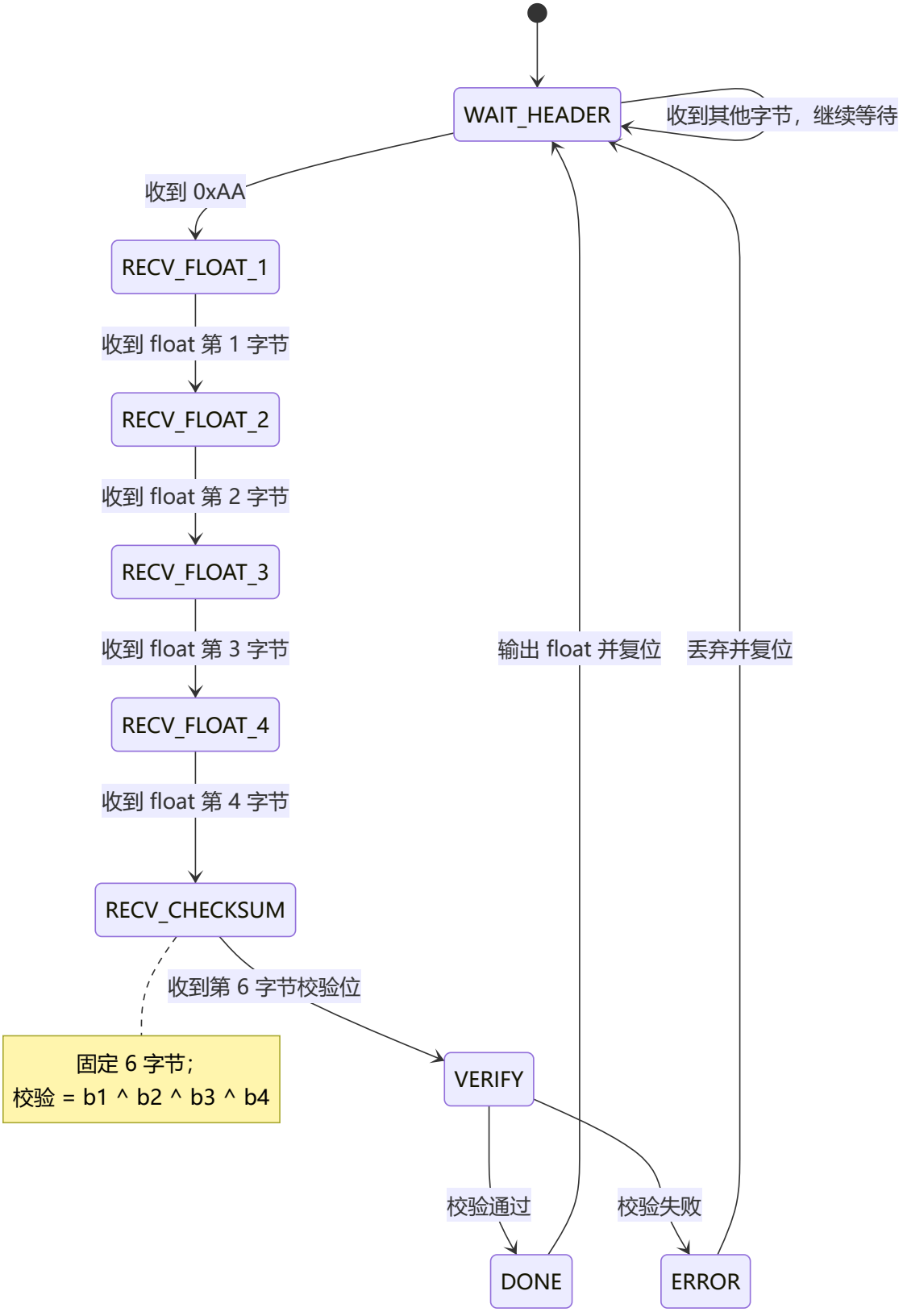
发送代码实现

```
void BLE_Send_Float(float num)
{
    uint8_t pack[6];
    pack[0] = 0xAA;
    memcpy(&pack[1], &num, 4);
```

```
    pack[5] = 0;
    for (int i = 1; i <= 4; i++)
        pack[5] ^= pack[i];
    for (int i = 0; i <= 5; i++)
    {
        while(USART_GetFlagStatus(USART2, USART_FLAG_TXE) == RESET);
        USART_SendData(USART2, pack[i]);
    }
}
```

接收状态机

让 DeepSeek 画了个解析包的状态机（咳咳）


WAIT_HEADER
收到其他字节，继续等待
收到 0xAA
RECV_FLOAT_1
收到 float 第 1 字节
RECV_FLOAT_2
收到 float 第 2 字节
RECV_FLOAT_3
收到 float 第 3 字节
RECV_FLOAT_4
收到 float 第 4 字节
RECV_CHECKSUM
收到第 6 字节校验位
固定 6 字节；
校验 = b1 ^ b2 ^ b3 ^ b4
VERIFY
校验通过
校验失败
DONE
ERROR
输出 float 并复位
丢弃并复位

知识点汇总

USART 串口初始化

IO 口初始化

- **TX 引脚 -> 复用推挽输出（GPIO_Mode_AF_PP）**
  - **“复用”**：STM32的引脚默认由 CPU 直接控制（通用 GPIO）。当设置为“复用”时，引脚的控制权就交给了 USART 外设，而不是 CPU 的 GPIO 数据寄存器。也就是说，我们不再通过写 GPIOA 的 ODR 寄存器来拉高拉低，而是由 USART 硬件自动在发送数据时拉高拉低。如果你不设置为复用，USART 根本无法控制这个引脚，串口发不出数据。
  - **“推挽”**：推挽输出拥有强大的驱动能力，可以主动输出高电平（通过内部 PMOS 管）和低电平（通过内部 NMOS 管），并且高低电平切换速度非常快。串口通信需要频繁、快速地在高低电平间切换以表示0和1，推挽输出能保证波形完美。如果配置为开漏输出，则必须依赖外部上拉电阻，上升沿会变缓，容易导致高速串口通信出错。
- **RX 引脚 -> 浮空输入（GPIO_Mode_IN_FLOATING）**
  - 输入模式有上拉、下拉、浮空。**浮空（Floating）意味着引脚内部既没有上拉电阻也没有下拉电阻，处于高阻态。**
  - 为什么选浮空？因为外部设备（蓝牙模块的 TX）会主动驱动这根线输出高低电平。如果STM32内部有上拉或下拉电阻，它就会和外部设备“较劲”，试图把电平拉到自己这边，这会干扰外部设备发出的信号，导致读取错误。浮空输入就是把控制权完全交给外部设备，STM32只负责“听”，不干预。
- **STATE 引脚 -> 上拉输入（GPIO_Mode_IPU）**
  - 由于 STATE 引脚在模块连接成功后通常会输出高电平（3.3V），连接断开时输出低电平（0V），**推荐配置为上拉输入（IPU）**，这样如果蓝牙模块没插好导致引脚悬空，STM32会默认读到高电平，不会产生随机乱跳。

串口初始化结构体

初始化结构体 `USART_InitTypeDef` 定义：

```
/**
  * @brief  USART Init Structure definition
  */

typedef struct
{
  uint32_t USART_BaudRate;            /*!< This member configures the USART
communication baud rate.
                                           The baud rate is computed using the
following formula:
                                            - IntegerDivider = ((PCLKx) / (16 *
(USART_InitStruct->USART_BaudRate)))
                                            - FractionalDivider =
((IntegerDivider - ((u32) IntegerDivider)) * 16) + 0.5 */

  uint16_t USART_WordLength;          /*!< Specifies the number of data bits
transmitted or received in a frame.
                                           This parameter can be a value of @ref
USART_Word_Length */
```

```
  uint16_t USART_StopBits;            /*!< Specifies the number of stop bits
transmitted.
                                           This parameter can be a value of @ref
USART_Stop_Bits */

  uint16_t USART_Parity;              /*!< Specifies the parity mode.
                                           This parameter can be a value of @ref
USART_Parity
                                           @note When parity is enabled, the
computed parity is inserted
                                                 at the MSB position of the
transmitted data (9th bit when
                                                 the word length is set to 9 data
bits; 8th bit when the
                                                 word length is set to 8 data
bits). */

  uint16_t USART_Mode;                /*!< Specifies wether the Receive or
Transmit mode is enabled or disabled.
                                           This parameter can be a value of @ref
USART_Mode */

  uint16_t USART_HardwareFlowControl; /*!< Specifies wether the hardware flow
control mode is enabled
                                           or disabled.
                                           This parameter can be a value of @ref
USART_Hardware_Flow_Control */
} USART_InitTypeDef;
```

波特率

成员	中文名称	含义
`USART_BaudRate`	波特率	决定通信速率。比如9600，代表每秒传输9600个二进制位。注释里的公式说明，单片机内部是通过你配置的波特率和当前外设总线频率（PCLK），自动计算并设置分频寄存器的。

波特率需要设置为与硬件一致，保证能正确的读取。

字长

成员	中文名称	含义
`USART_WordLength`	字长/数据位	每一帧数据包含的有效数据位数。通常是8位（`0x0000`），也可以配置为9位（`0x1000`）。

只有这两种选择：

```
#define USART_WordLength_8b                  ((uint16_t)0x0000)
#define USART_WordLength_9b                  ((uint16_t)0x1000)
```

停止位

成员	中文名称	含义
`USART_StopBits`	停止位	一帧数据的结束标志。通常是1位（`0x0000`），也可以配0.5、1.5、2位，用于兼容老式设备。

如下四种选择：

```
#define USART_StopBits_1                     ((uint16_t)0x0000)
#define USART_StopBits_0_5                   ((uint16_t)0x1000)
#define USART_StopBits_2                     ((uint16_t)0x2000)
#define USART_StopBits_1_5                   ((uint16_t)0x3000)
```

校验位

成员	中文名称	含义
`USART_Parity`	校验位	用于检测数据传输过程中是否发生位翻转错误。`No`（无）、`Even`（偶校验）、`Odd`（奇校验）。

三种选择：

```
#define USART_Parity_No                      ((uint16_t)0x0000)
#define USART_Parity_Even                    ((uint16_t)0x0400)
#define USART_Parity_Odd                     ((uint16_t)0x0600)
```

模式

成员	中文名称	含义
`USART_Mode`	模式	控制收发方向。`Rx`（接收）、`Tx`（发送）。两者用按位或 `|` 连接表示同时开启。

两种选择：

```
#define USART_Mode_Rx                        ((uint16_t)0x0004)
#define USART_Mode_Tx                        ((uint16_t)0x0008)
```

流控

成员	中文名称	含义
`USART_HardwareFlowControl`	硬件流控	在串口通信或计算机与单片机通信中，流控用于防止数据丢失。当接收端缓冲区已满时，发送端会收到“停止发送”的信号，待接收端准备好后再继续传输。

- **无流控（None）**：发送方不管接收方死活，自顾自地发。**蓝牙透传通常用这个。**
  - 如果前面的数据还没传出去，新的数据就来了，那么新的数据会覆盖掉旧的数据，造成一些数据丢失。
- **硬件流控（RTS/CTS）**：
  - **RTS（Request To Send，请求发送）**：**本设备输出。**它拉低表示“我准备好接收了，你发吧”。它拉高表示“我缓冲区满了，你停一下”。
  - **CTS（Clear To Send，清除发送）**：**本设备输入。**它拉低表示“对方准备好接收了，我可以发数据了”。
  - 在串口通信中，硬件流控会额外占用两根物理引脚（RTS和CTS）。如果你的板子没有引出这两根线，**绝对不要开流控**，否则串口会一直死等，看起来像没反应。
- **软件流控（XON/XOFF）**：通过发送特殊字符（比如 `0x11` 和 `0x13`）来通知对方暂停或继续。STM32 的这个结构体里不直接配置软件流控，通常由应用层代码自己处理。

硬件流控选择如下：

```
#define USART_HardwareFlowControl_None       ((uint16_t)0x0000)
#define USART_HardwareFlowControl_RTS        ((uint16_t)0x0100)
#define USART_HardwareFlowControl_CTS        ((uint16_t)0x0200)
#define USART_HardwareFlowControl_RTS_CTS    ((uint16_t)0x0300)
```

USART 串口输出

> 数据寄存器（USART_DR）是串口通信中数据交换的唯一通道，状态寄存器（USART_SR）是串口实时监控与事件触发的核心。

`USART_SR` 是一个只读寄存器，其每一位都反映了USART硬件当前的某个特定状态。它是软件感知硬件行为、做出决策的唯一信息源，其重要性不亚于控制寄存器。

- **位5（RXNE - Read Data Register Not Empty）**：接收数据寄存器非空标志。当硬件将一帧完整的数据从 RSR 转移到 RDR 后，`RXNE` 被硬件自动置1。 **工程目的**：指示有新数据可供 CPU 读取。
- **位6（TC - Transmission Complete）**：发送完成标志。当 TDR 与 TSR 均为空，且最后一比特的停止位已被发送完毕时，`TC` 被硬件置1。 **工程目的**：指示整个发送过程的终结。
  - 在 USART_DR 寄存器中写入最后一个数据字后，要等待 TC=1，它表示最后一个数据帧的传输结束。当需要关闭 USART 或需要进入停机模式之前，需要确认传输结束，避免破坏最后一次传输。
- **位7（TXE - Transmit Data Register Empty）**：发送数据寄存器空标志。当 TDR 被硬件清空（即数据已从 TDR 转移到 TSR）后，`TXE` 被置1。 **工程目的**：指示可以向 `DR` 写入下一个字节。

`USART_DR` 是一个32位的双功能寄存器，其物理地址相同，但根据读/写操作的不同，访问的是内部不同的子寄存器：写操作访问发送数据寄存器（TDR），读操作访问接收数据寄存器（RDR）。这种设计是典型的“统一地址、分时复用”思想，极大地简化了编程模型。

- 向 `DR` 写入数据，并非立即将其发送出去。数据首先被锁存到发送数据寄存器（TDR）中，随后在移位寄存器（TSR）空闲时，由硬件自动将其从 TDR 转移到 TSR，并开始串行移位输出。同理，接收到的数据首先进入接收移位寄存器（RSR），待一帧接收完毕，再由硬件将其从 RSR 转移到接收数据寄存器（RDR）

USART 的 DR 寄存器只有一个字节的缓冲深度。如果你连续调用 `USART_SendData` 两次，而第一次的数据还没被硬件移走（TXE=0），第二次写入就会**直接覆盖**第一次的数据，导致第一个字节永远发不出去。

所以才需要第一步：循环等待 TXE 位被置1。

第二步：发送数据。

```
/**
  * @brief  Transmits single data through the USARTx peripheral.
  * @param  USARTx: Select the USART or the UART peripheral.
  *   This parameter can be one of the following values:
  *   USART1, USART2, USART3, UART4 or UART5.
  * @param  Data: the data to transmit.
  * @retval None
  */
void USART_SendData(USART_TypeDef* USARTx, uint16_t Data)
{
  /* Check the parameters */
  assert_param(IS_USART_ALL_PERIPH(USARTx));
  assert_param(IS_USART_DATA(Data));

  /* Transmit Data */
  USARTx->DR = (Data & (uint16_t)0x01FF);
}
```

别看这个 `Data` 是十六位的，实际上有用的也就8~9位，具体看自己的设置。

因为 STM32的 USART 数据寄存器（DR）物理上也只有**9个有效位**（第0~8位）。这9位怎么用，取决于你的配置：

- 如果配置为**8位数据 + 1位校验**：第0~7位是实际数据，第8位是硬件自动生成的校验位。
- 如果配置为**9位数据 + 无校验**：第0~8位全部是实际数据（最多可以传512个值）。
  库函数中使用了 `& 0x01FF` 防止你传入超过9位的数据，导致寄存器高位被污染。

所以说一次只能发送**8或9位**的数据，而不是16位。

NVIC 简介

> 好像计组课上会讲是不是……

NVIC（Nested Vectored Interrupt Controller），嵌套向量中断控制器。它是 ARM Cortex-M 系列微控制器内核中集成的一个关键组件，用于管理中断和异常。

NVIC 与处理器内核紧密耦合，可实现低延迟的中断处理。其核心功能包括中断优先级管理、支持中断嵌套、通过中断向量表确定中断服务程序入口等。不同架构的 Cortex-M 处理器在 NVIC 支持的中断数量、优先级等级等方面存在差异。它还支持尾链、晚到等机制以优化中断响应时间。

中断是抢占式的——CPU 正在执行任何代码时，都可能被硬件中断打断，跳转去执行 ISR 。

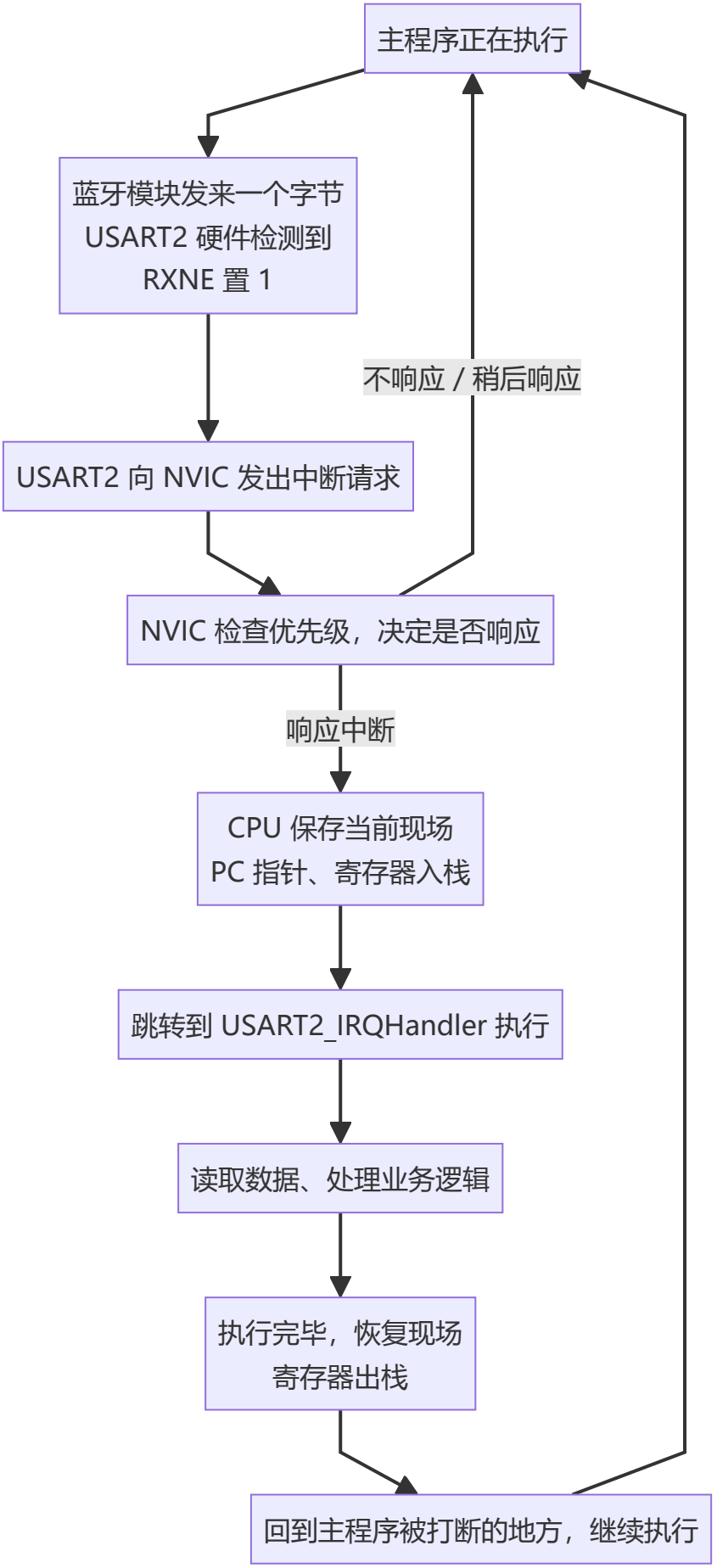
关于优先级

- **抢占优先级**：决定能否打断正在执行的其他中断。数值越小，优先级越高。比如抢占优先级为0的中断可以打断抢占优先级为1的中断。
- **子优先级**：当两个中断同时挂起（抢占优先级相同）时，子优先级决定谁先执行。
- **在无人机项目中**：飞控的电机控制中断应该设最高抢占优先级，蓝牙接收中断设较低优先级。这样即使蓝牙正在处理数据，电机控制也能立刻打断它。

中断服务函数的注意事项

1. **函数名不能改**：`USART2_IRQHandler` 这个名字是启动文件（startup_stm32f10x_md.s）里写死的，写错了就进不了中断。
2. **执行时间要短**：ISR 里不要放延时、浮点运算等耗时操作。应该只做“读取数据、存到缓冲区、置个标志位”这些快速操作，把复杂处理留给主循环。
3. **清除标志位**：对于 RXNE ，读取 DR 寄存器会自动清除标志。但其他中断（如 TC、ORE）需要手动清除。

中断的完整流程


主程序正在执行
蓝牙模块发来一个字节
USART2 硬件检测到
RXNE 置 1
不响应 / 稍后响应
USART2 向 NVIC 发出中断请求
NVIC 检查优先级，决定是否响应
响应中断
CPU 保存当前现场
PC 指针、寄存器入栈
跳转到 USART2_IRQHandler 执行
读取数据、处理业务逻辑
执行完毕，恢复现场
寄存器出栈
回到主程序被打断的地方，继续执行